


# AIDLY

## A Venue Operations Layer for Crowd Emergencies

**Software Engineering Project Report**

SUBMITTED BY

| | |
|---|---|
| Sourav Kumar | 24/48044 |
| Janvi Monga | 24/48092 |
| Avijit Sakkarwal | 24/48079 |

Submitted to

Uma Ma'am

Subject

Software Engineering

# Contents


| | | |
|---|---|---|
| **1** | **Problem Statement** | **4** |
| 1.1 | Overview | 4 |
| 1.2 | Why the Present Method Fails | 4 |
| 1.3 | Challenges in the Current System | 5 |
| 1.4 | The Proposed Solution | 5 |
| 1.5 | Stakeholders | 5 |
| 1.6 | Scope of the System | 6 |
| **2** | **Requirements Identification** | **7** |
| 2.1 | Method of Elicitation | 7 |
| 2.2 | The Prototype | 7 |
| 2.3 | Walkthrough of the Reporting Path | 7 |
| 2.4 | Specification of the Reporting Inputs | 9 |
| 2.5 | The Control Surfaces | 10 |
| 2.6 | Findings Carried into the Requirements | 10 |
| 2.7 | Functional Requirements | 11 |
| 2.8 | Nonfunctional Requirements | 12 |
| 2.9 | Requirements Traceability | 13 |
| **3** | **Process Model** | **14** |
| 3.1 | Candidate Models | 14 |
| 3.2 | Characteristics of This Project | 14 |
| 3.3 | Comparison of the Candidates | 14 |
| 3.4 | The Selected Model | 15 |
| 3.5 | Reasons for the Selection | 15 |
| 3.6 | Why Each Alternative Was Rejected | 15 |
| 3.7 | The Increment Plan | 16 |
| **4** | **Data Flow Diagrams and Data Dictionary** | **17** |
| 4.1 | Notation | 17 |
| 4.2 | Level 0: Context Diagram | 17 |
| 4.3 | Level 1 Data Flow Diagram | 20 |
| 4.4 | Reading the Level 1 Diagram | 21 |
| 4.5 | Process Specifications | 22 |
| 4.6 | Data Dictionary: Data Flows | 24 |
| 4.7 | Data Dictionary: Data Stores | 24 |
| 4.8 | Data Dictionary: Data Elements | 25 |

| | | |
|---|---|---|
| **5** | **Use Case Diagram** | **27** |
| 5.1 | Actors | 27 |
| 5.2 | The Diagram | 28 |
| 5.3 | Use Case Inventory | 29 |
| **6** | **Use Case Templates** | **30** |
| 6.1 | UC 01: Submit Emergency Report | 30 |
| 6.2 | UC 02: Dispatch Responder | 31 |
| 6.3 | UC 03: Authenticate Control Room Operator | 32 |
| 6.4 | Incident State Transitions | 33 |
| **7** | **Login Module Connection with Database** | **34** |
| 7.1 | What Was Built | 34 |
| 7.2 | Structure of the Module | 34 |
| 7.3 | The Users Table | 35 |
| 7.4 | The Connection Layer | 36 |
| 7.5 | Password Hashing | 36 |
| 7.6 | Access Tokens | 37 |
| 7.7 | The Authentication Service | 37 |
| 7.8 | The Endpoints | 38 |
| 7.9 | The Interface Offered by the Module | 39 |
| 7.10 | The Sign In Screen | 40 |
| 7.11 | Sequence of One Authentication | 41 |
| 7.12 | Execution Against the Database | 42 |
| 7.13 | Verification | 43 |
| **8** | **References** | **45** |

# 1
# Problem Statement

## 1.1 Overview

Large public gatherings are ordinary events in India. A college festival, a stadium fixture, a religious procession or a concert can hold tens of thousands of people inside a single boundary for several hours. Almost every such venue already employs the people needed to handle an emergency: marshals, medical staff, fire wardens, and a control room that supervises them. What no venue reliably has is a channel that carries what an attendee can see to the people who are paid to act on it.

Aidly exists to supply that channel. It is a venue operations layer: software that belongs to one venue, is used by the people inside it, and is answered by that venue's own responders.

## 1.2 Why the Present Method Fails

When something goes wrong in a dense crowd, the accepted way to summon help is a voice call to an emergency number. That method fails at exactly the moment it is needed most, and the failure is structural rather than occasional. Three causes appear together, in the same conditions, and each one makes the others worse.

| Cause | What happens | Consequence |
|---|---|---|
| Acoustic | A crowd generates continuous noise across the whole speech range. The caller cannot hear the operator and the operator cannot hear the caller. | The two facts that matter, what has happened and where, are the least likely to survive the call. |
| Network | Thousands of handsets contend for one cell. Voice circuits are refused or dropped when the largest number of people are trying to place a call. | The service degrades in proportion to the size of the emergency, which is the opposite of what is required. |
| Informational | A call carries one person's account, once, to one listener. | Thirty witnesses to one crush produce thirty conversations, so the scale of the event stays invisible at the moment it must be judged. |

*Table 1.1 Three simultaneous causes of failure in the present method*

A short structured message behaves differently under all three conditions. It needs no speech, it is a few hundred bytes rather than a held circuit, and it arrives as data that can be counted, compared and merged with other messages describing the same event.

## 1.3 Challenges in the Current System

1. **The venue is not the recipient.** India operates the Emergency Response Support System, reached on 112. It handles the path from a citizen to the state well, and Aidly does not attempt to replace it. But 112 routes a caller to a state control room. It does not give one particular venue a live picture of its own ground, it does not dispatch that venue's own marshals and medics, and it cannot push exit guidance to the people standing inside that venue.
2. **Proximity is wasted.** At a festival or a stadium, trained responders are frequently within a hundred metres of an incident. Nothing routes a report to them, resolves where it came from, or tells the control room that thirty messages describe one event rather than thirty.
3. **Duplicate reports are not recognised as duplicates.** Without merging, the control room cannot distinguish one serious incident with many witnesses from many small unrelated ones.
4. **Location is assumed to be available.** Systems that treat a missing device position as an error become unusable in exactly the conditions they were built for. Indoor venues, dense steel structures and saturated networks all deny position routinely.
5. **Some reports cannot be made openly.** Harassment at crowded events is common and heavily under reported. A loud or visibly obvious alert can increase the risk to the person reporting rather than reduce it, so a system offering only a conspicuous alarm will not be used by the people who most need it.
6. **Nothing is accountable afterwards.** A phone call leaves no record that the venue holds. There is no way to establish who was told, when, who responded, or how long it took.

## 1.4 The Proposed Solution

Anyone inside a venue reports in a single tap, optionally attaching a short description or a photograph. The system resolves the report to a named zone, classifies it, merges it with other reports describing the same event, and presents the control room with a live and deduplicated picture of its own ground. The operator dispatches the venue's own responders, tracks acknowledgement and arrival, and broadcasts guidance to attendees where that is needed.

Two design commitments follow from the analysis above and govern every requirement in this report.

- **The network is unreliable exactly when the system matters.** Where location services fail, the reporter chooses the nearest gate and the report proceeds. Nothing in the reporting path may block on an optional input.
- **Aidly escalates to 112, it does not compete with it.** The national service sits outside the system boundary and receives a deliberate handoff initiated by the control room.

## 1.5 Stakeholders

Six parties have an interest in the system, and their interests differ enough that a design decision which suits one can work against another.

| Stakeholder | Interest in the system |
|---|---|
| Attendee | Wants help to arrive, and wants reporting to take one action rather than a conversation. May be unable to speak or to be seen reporting. |
| Control room operator | Needs one ranked picture of the ground rather than a queue of unrelated messages, and needs to know which responder is where. |
| Responder | Needs an assignment with a location they can navigate to, and a way to acknowledge it so the control room stops chasing. |
| Venue administrator | Owns the configuration: the zones, the roster, and the record that survives the event. |
| Venue management | Answerable afterwards. Needs an account of what was reported, when it was acted on, and by whom. |
| ERSS 112 | Receives escalations that exceed what the venue can handle, and benefits from receiving one consolidated incident rather than many calls. |

*Table 1.2 Stakeholders and their interests*

## 1.6 Scope of the System

The boundary is stated in both directions, because what a system will not do is as much a decision as what it will.

| Included | Excluded |
|---|---|
| A single venue divided into named zones, configured by an administrator. | Any claim to replace or duplicate the national 112 service. |
| Four human actors: reporter, responder, control room operator and venue administrator. | Programmatic integration with the ERSS backend, for which no public interface is published. |
| Categories covering fire, medical, crowd, accident, violence, harassment and other. | Dedicated hardware such as wearables or fixed panic buttons. |
| A mobile first web application needing no installation by an attendee. | Nationwide consumer deployment outside a venue that runs the system. |
| Live updates to operator and responder screens without polling. | Locating a handset through the telecom carrier. |
| An append only record of every incident and every access to evidence. | Long term storage of photographic evidence beyond the stated retention period. |

*Table 1.3 Scope boundary*

# 2
# Requirements Identification

## 2.1 Method of Elicitation

Requirements for this system could not be gathered by interview alone. The behaviour under discussion happens in conditions that are hard to describe from memory, and the people who would use the reporting screen are ordinary attendees rather than trained operators. Asking someone what they would do in a crush produces a considered answer, and a considered answer is not what the system will receive.

The team therefore built a working prototype first and used it as the instrument of elicitation. Testers were given a device and a scenario and were watched. Every requirement in this chapter is traceable either to something a tester did, or to an explicit design decision recorded during the analysis that followed.

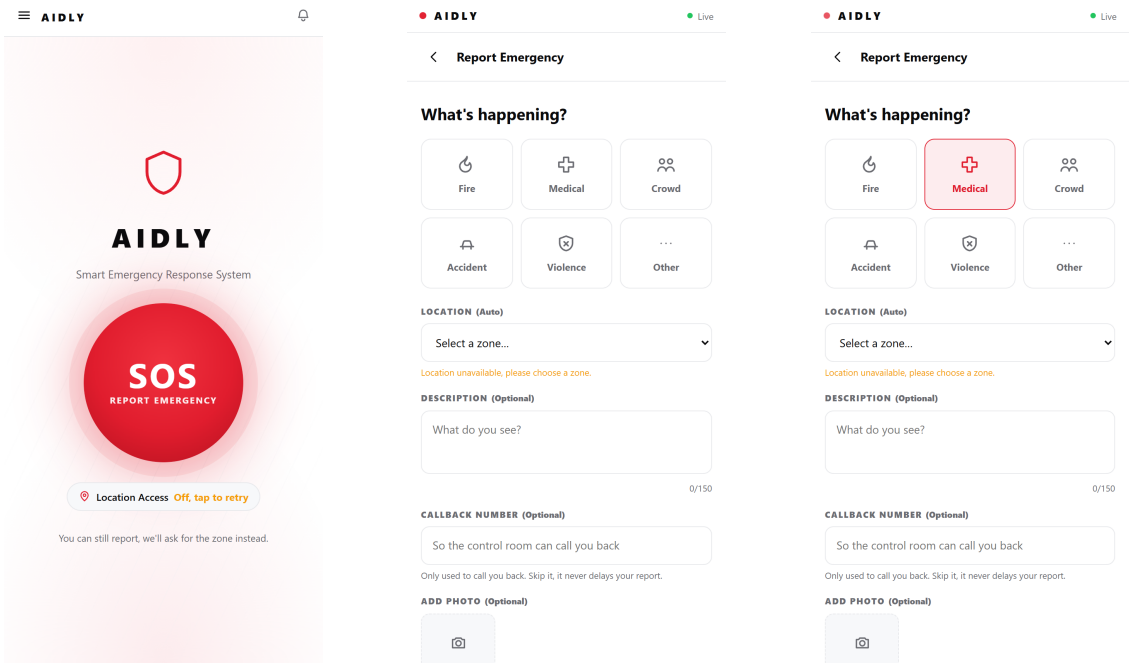
The prototype is a throwaway. It was built to answer questions about the interface and the vocabulary, and its code is not intended to become the delivered product. What survives from it is the requirement set below.

## 2.2 The Prototype

The prototype implements the reporter facing path end to end against a live backend: one prominent action on the landing screen, a category chooser, a location control that falls back to a gate list, an optional description, photograph and callback number, and after submission the progress of triage and then live tracking of the incident.

## 2.3 Walkthrough of the Reporting Path

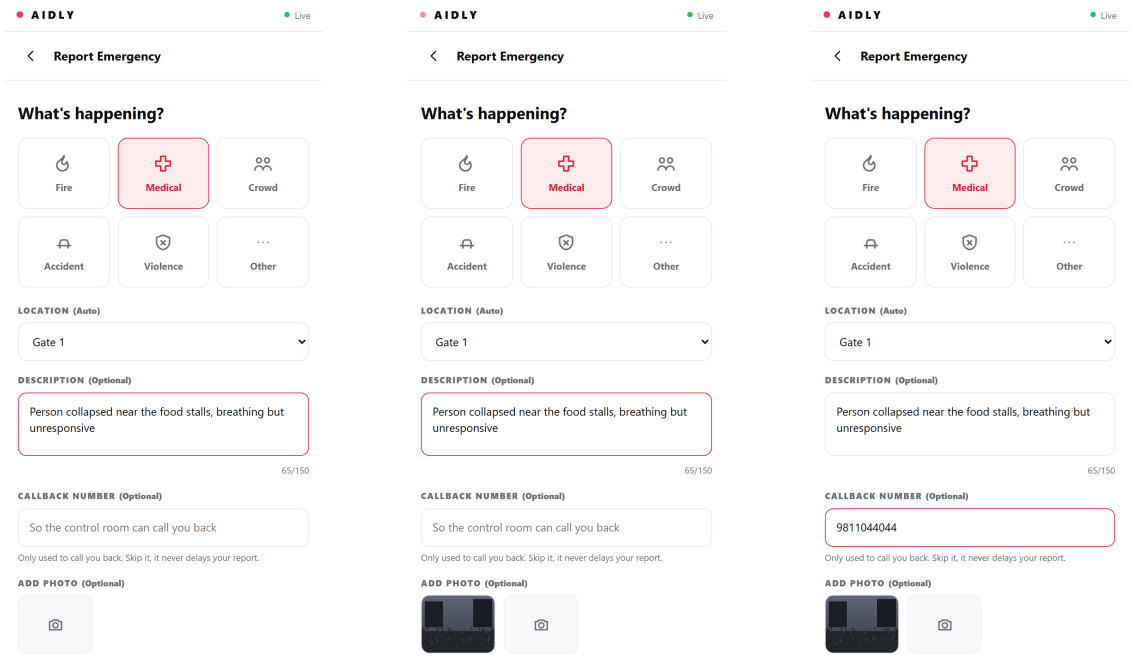
The nine screens below are the complete path an attendee travels.




1. Landing screen | 2. Category chooser | 3. Category selected

*Figure 2.1 Opening the report. One action dominates the landing screen. Selecting a category is the only mandatory input.*

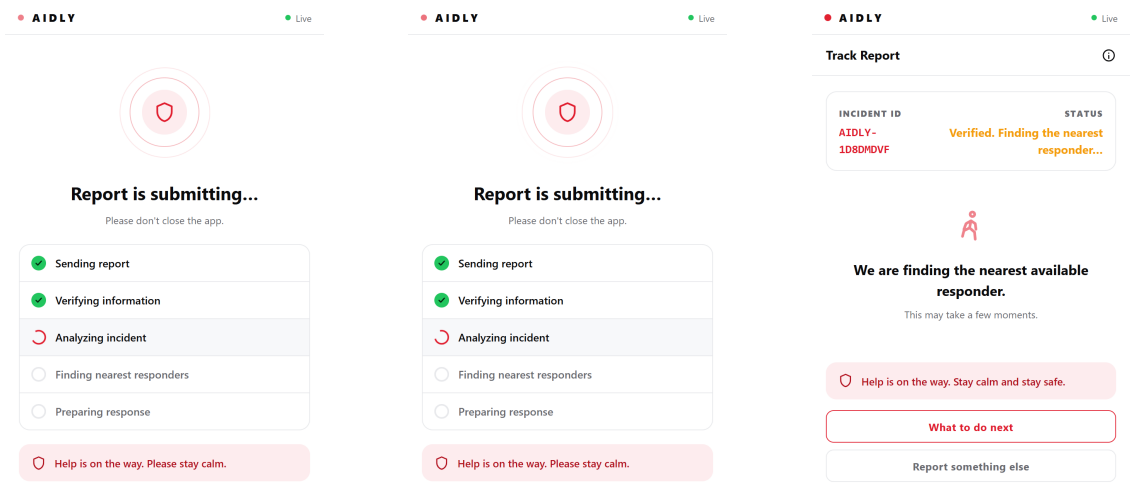
The landing screen carries a single control. Early layouts offered several entry points and testers hesitated over which to use, so everything except the reporting action was removed. The notice beneath it states plainly that a report can still be filed without location, which prevents a tester from abandoning the screen when the position badge shows as unavailable.




4. Optional description | 5. Photograph attached | 6. Callback number

*Figure 2.2 The optional inputs. Each is skippable, and each states why it exists.*

The description carries a visible budget of 150 characters. An unbounded field produced long narrative entries that delayed submission without adding operational detail. The callback field states that location is taken from the device and never from the number, which removed the single most common objection raised during testing.




7. Submission pipeline 8. Guidance returned 9. Live tracking

*Figure 2.3 After submission. The pipeline is shown step by step, then the report becomes a tracked incident with an identifier.*

Showing the pipeline rather than a single spinner was a direct result of testing. Testers who saw an undifferentiated wait assumed the report had failed and pressed the control again, producing duplicates. Naming each stage as it completes removed that behaviour. The tracking screen then gives the reporter an incident identifier and a live status, so the question that follows every emergency report, namely whether anyone is actually coming, has a visible answer.

## 2.4 Specification of the Reporting Inputs

Table 2.1 specifies each input on the reporting screen. The required column is the operative one: only the category is required, because NFR 03 forbids any optional input from preventing a submission.

| Input | Type | Required | Rule applied | Behaviour if absent |
|---|---|---|---|---|
| Category | One of seven | Yes | Exactly one selection | Submission control stays disabled |
| Zone | Reference | Yes | Device position, else a chosen gate | Gate list is presented, report proceeds |
| Description | Text | No | At most 150 characters, counter shown | Report proceeds with no description |
| Photograph | Image file | No | Uploaded, reference stored on the report | Report proceeds with no evidence |

| Input | Type | Required | Rule applied | Behaviour if absent |
|---|---|---|---|---|
| Callback number | Digits | No | Ten digits, non digits discarded | Report proceeds, control room cannot ring back |
| Discreet flag | Boolean | No | Set for personal safety categories | Ordinary confirmation is shown |

*Table 2.1 Specification of the reporting inputs*

Two rules in that table were not obvious before the prototype was observed. Discarding non digit characters from the callback number removed a class of failed submissions caused by testers typing spaces and country codes. Keeping the submission control disabled until a category is chosen proved better than allowing submission and then reporting an error, because a rejected emergency report is read as a failure of the system rather than as a correction.

## 2.5 The Control Surfaces

Two further surfaces exist for staff. Both are protected, which is itself a requirement: the reporting path is deliberately public, and everything operational is deliberately not.

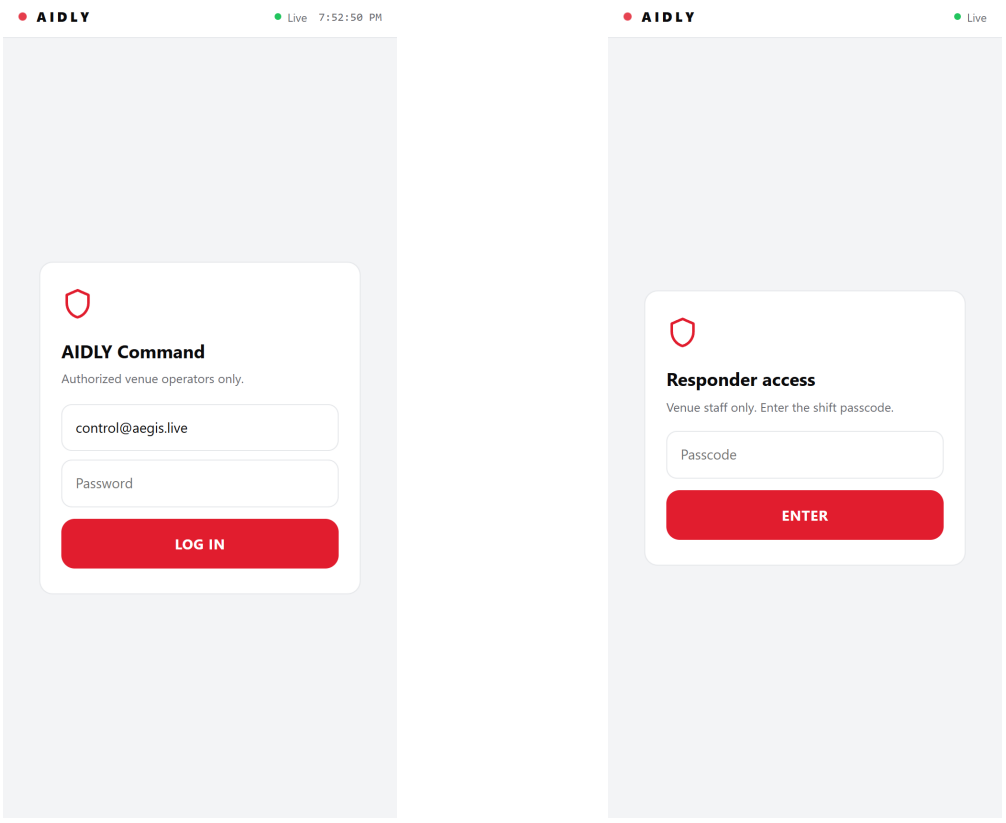



*Figure 2.4 The two protected entry points. Chapter 7 implements authentication of this kind against a database.*

## 2.6 Findings Carried into the Requirements

1. **One action must dominate the landing screen.** Choice at the entry point produced hesitation in every session.

2. **Location must never block a report.** The browser refused to supply a position in a large share of trials. Treating that as an error produced an unusable screen, so the gate list was introduced and the report proceeds.
3. **Six categories are enough and are understood.** Testers selected the intended category without instruction. A longer list slowed selection without improving accuracy.
4. **Progress must be itemised, not implied.** An undifferentiated wait caused repeat submissions and therefore duplicate reports.
5. **Optional fields must say why they exist.** Testers refused the callback number until the screen explained it is not used to locate them.
6. **A reporter needs to see that something happened.** An incident identifier and a live status changed how testers described the experience more than any other change.

## 2.7 Functional Requirements

Eighteen functional requirements follow. Each is traceable either to a finding recorded above or to the analysis that followed the prototype.

| ID | Requirement | Source |
|---|---|---|
| FR 01 | The system shall allow any attendee to submit a report in a single action, without registration or login. | Finding 1 |
| FR 02 | The system shall let the reporter classify the report into one of the defined categories. | Finding 3 |
| FR 03 | The system shall resolve every report to a named zone, using the device position where available and a reporter chosen gate otherwise. | Finding 2 |
| FR 04 | The system shall accept an optional description of at most 150 characters. | Finding 5 |
| FR 05 | The system shall accept an optional photograph and store it against the report. | Prototype |
| FR 06 | The system shall accept an optional callback number and shall never use it to determine location. | Finding 5 |
| FR 07 | The system shall report the progress of submission as itemised stages. | Finding 4 |
| FR 08 | The system shall issue an incident identifier and show a live status to the reporter. | Finding 6 |
| FR 09 | The system shall assign a severity tier to every incident and record which method produced it. | Analysis |
| FR 10 | The system shall merge reports of the same category, in the same zone, within a configured time window into one incident, and maintain a corroboration count. | Analysis |
| FR 11 | The system shall dispatch the nearest available responder whose role and attributes match the incident category. | Analysis |
| FR 12 | The system shall escalate an unacknowledged high severity incident automatically after a configured interval. | Analysis |
| FR 13 | The system shall provide an escalation handoff to ERSS 112 initiated by the control room. | Analysis |
| FR 14 | The system shall allow a personal safety report to be submitted discreetly, producing no sound, no vibration and no visible change of screen state. | Analysis |
| FR 15 | The system shall authenticate operators, responders and administrators before granting access to any operational screen. | Section 2.4 |
| FR 16 | The system shall allow the control room to broadcast advisories and exit guidance to attendees. | Analysis |

| ID | Requirement | Source |
|---|---|---|
| FR 17 | The system shall maintain an append only record of every state change and every access to attached evidence. | Analysis |
| FR 18 | The system shall allow a venue administrator to define the venue, its named zones and the responder roster, including the role and attributes of each responder. | Analysis |

*Table 2.2 Functional requirements*

## 2.8 Nonfunctional Requirements

The requirements below constrain how the system must behave rather than what it must do.

| ID | Category | Requirement |
|---|---|---|
| NFR 01 | Performance | A submitted report shall appear on the control room screen within two seconds under normal network conditions. |
| NFR 02 | Reliability | The reporting path shall remain usable when the geolocation service is unavailable or refused. |
| NFR 03 | Reliability | No optional input shall be able to prevent a report from being submitted. |
| NFR 04 | Usability | An attendee with no prior exposure shall be able to submit a correctly categorised report without instruction. |
| NFR 05 | Security | Passwords shall be stored only as salted one way hashes. A plain password shall never be written to storage or to any log. |
| NFR 06 | Security | A failed sign in shall return the same message and take comparable time regardless of whether the account exists. |
| NFR 07 | Security | An account that has been deactivated shall lose access on its next request, not when its token expires. |
| NFR 08 | Privacy | The identity of a reporter shall be visible to the control room and shall not be exposed to responders who do not require it. |
| NFR 09 | Portability | The system shall run on hardware the venue controls, with no dependence on a managed third party service. |
| NFR 10 | Auditability | Every incident shall be reconstructible from the recorded events alone. |

*Table 2.3 Nonfunctional requirements*

## 2.9 Requirements Traceability

Every functional requirement is carried into the later chapters. Table 2.4 records where each is realised, so that no requirement is left without a design and no part of the design exists without a requirement behind it.

| Requirement | Process | Use case | Built |
|---|---|---|---|
| FR 01 to FR 04, FR 06 to FR 08 | 1.0 Report Intake | UC 01 | Prototype |
| FR 05 | 1.0 Report Intake | UC 05, included by UC 01 | Prototype |
| FR 09 | 2.0 Triage | UC 06 | No |
| FR 10 | 3.0 Merge and Corroborate | UC 06 | No |
| FR 11 | 4.0 Dispatch | UC 02 | No |
| FR 12 | 5.0 Escalation | UC 07 | No |
| FR 13 | 5.0 Escalation | UC 08 | No |
| FR 14 | 1.0 Report Intake | UC 04 | No |
| FR 15 | Cross cutting, see Section 4.7 | UC 03 | **Yes, Chapter 7** |
| FR 18 | 8.0 Venue Configuration | UC 10 | No |
| FR 16 | 6.0 Broadcast | UC 09 | No |
| FR 17 | 7.0 Audit and Evidence | UC 08 | No |

*Table 2.4 Requirements traceability matrix*

# 3
# Process Model

## 3.1 Candidate Models

Five process models were considered: the Waterfall model, the Prototyping model, the Spiral model, the Agile approach represented by Scrum, and the Incremental model. The choice between them follows from the characteristics of this project rather than from a general preference, so those characteristics are stated first.

## 3.2 Characteristics of This Project

- **The core requirements are already known.** A prototype has been built and evaluated, and Chapter 2 records a stable requirement set drawn from observed behaviour rather than speculation.
- **The system must be demonstrable at every stage.** There are fixed review points, so a model that produces nothing runnable until late carries unacceptable risk.
- **A partly built emergency system is worse than none.** Anything that invites a person to report and then fails to deliver help is actively harmful, so each stage must end with the whole path working rather than with a half connected feature.
- **The functions are separable.** Reporting, triage, dispatch, escalation and broadcast are distinct and can be delivered one after another.
- **The team is small and works together.** Three members, with no dedicated customer representative available on demand.

## 3.3 Comparison of the Candidates

Each candidate is judged against the five characteristics above.

| Model | Fit | Deciding factor |
|---|---|---|
| Waterfall | Poor | Assumes requirements are frozen and produces nothing executable until late. A change discovered during construction is expensive to absorb, and no early stage would be demonstrable. |
| Prototyping | Partial | Already used, and used correctly, to elicit requirements. It is a technique inside the analysis stage here rather than the model for the whole build. |
| Spiral | Poor | Its strength is repeated formal risk analysis, which suits large and costly systems. On a project of this size the analysis effort would exceed the construction effort. |
| Scrum | Partial | Attractive for its iteration, but it presumes a product owner available continuously to reprioritise a backlog, and it deliberately de emphasises the fixed documentation this project must produce. |
| Incremental | Strong | Delivers the system in functional increments, each testable and demonstrable, while keeping the documented analysis and design stages intact. |

| Model | Fit | Deciding factor |
|---|---|---|

*Table 3.1 Comparison of candidate process models*

## 3.4 The Selected Model

The **Incremental model** is selected. Requirements are established once and then partitioned. Each increment passes through analysis, design, coding and testing, and each delivers a working addition to the product. The first increment produces a usable core, and every increment after it extends that core without breaking it.

## 3.5 Reasons for the Selection

1. **The requirements are stable enough to partition.** The prototype settled the questions that would otherwise have forced continuous renegotiation. That stability is the precondition the Incremental model assumes, and this project satisfies it.
2. **Every increment leaves a working system.** This matters more here than in an ordinary project, for the reason given in Section 3.2. Each increment is defined to end at a point where the path from report to resolution still functions.
3. **Risk is reduced by ordering rather than by analysis.** The first increment carries the highest technical risk: the database, the schema and authentication. Putting it first means the riskiest work is proven while there is still time to respond to what it reveals. That is what Chapter 7 reports.
4. **It preserves the documented stages.** The analysis and design work in Chapters 4 to 6 is produced once and then refined, rather than being replaced by continuous renegotiation.
5. **It suits a team of three.** Increments can be assigned as whole vertical slices, so each member owns a complete function rather than a fragment of one.

## 3.6 Why Each Alternative Was Rejected

### 3.6.1 Waterfall

Waterfall requires requirements to be complete and frozen before design begins, and produces no executable output until construction is well advanced. This project has already changed direction once in response to external review, and a model with no cheap route for absorbing such a change is the wrong instrument. The absence of an early running system would also leave defects undiscovered until late, which is precisely what happened with the hashing dependency described in Section 7.12.

### 3.6.2 Spiral

Spiral is built around repeated risk analysis at every cycle. That machinery earns its cost on large, expensive and long running systems. Here the identified risks are few and are adequately handled by the ordering of increments, so the ceremony would consume effort that the construction needs.

### 3.6.3 Agile and Scrum

Scrum was the closest competitor and was rejected for two specific reasons. It assumes a product owner continuously available to reprioritise a backlog, and no such role exists on this team. It also values working software above comprehensive documentation, whereas this project must produce a fixed analysis and design record. The Incremental model supplies the iteration that made Scrum attractive without discarding that record.

### 3.6.4 Prototyping as the whole model

Prototyping has already served its purpose in Chapter 2. Retaining it as the model for the entire build would risk the classic failure of that approach, in which an exploratory prototype is progressively patched into a production system and carries its shortcuts forward. The prototype here is explicitly a throwaway, and the rebuild described in Chapter 7 shares none of its code.

## 3.7 The Increment Plan

The requirement set is partitioned into five increments, ordered so that the riskiest work is proven first.

| Increment | Delivered function | Leaves working |
|---|---|---|
| I | Database schema, connection layer, and authentication for operators, responders and administrators. | A system a real user can sign into. |
| II | Report intake, zone resolution and persistence. | An attendee can file a report that is stored and retrievable. |
| III | Triage, merging and the control room incident view. | The control room sees merged incidents with corroboration. |
| IV | Dispatch, acknowledgement, resolution and escalation. | The full path from report to resolution operates. |
| V | Broadcast, discreet reporting, and the audit and evidence record. | The requirement set of Chapter 2 is met. |

*Table 3.2 Increment plan*

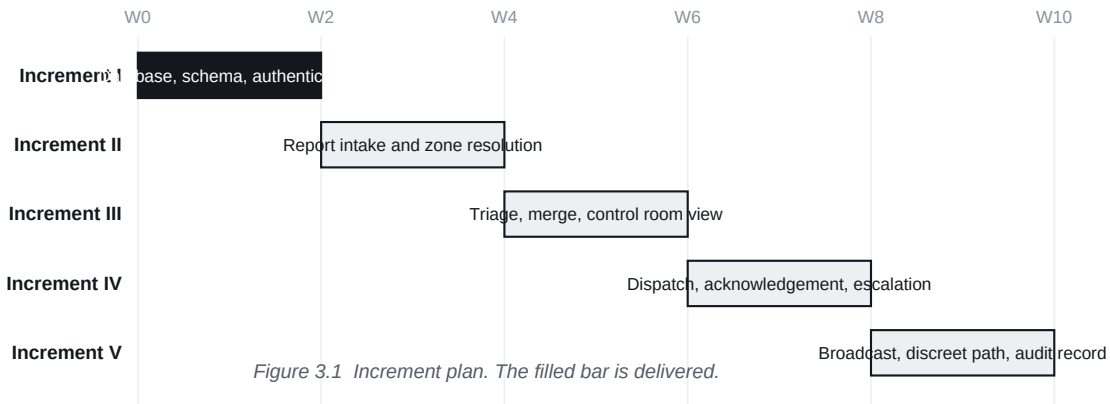



*Figure 3.1 Increment plan. The filled bar is delivered.*

# 4
# Data Flow Diagrams and Data Dictionary

## 4.1 Notation

Figure 4.1 gives the notation. A data flow is labelled for the data in motion rather than for the action being performed, which is what keeps a data flow diagram distinct from a flowchart.

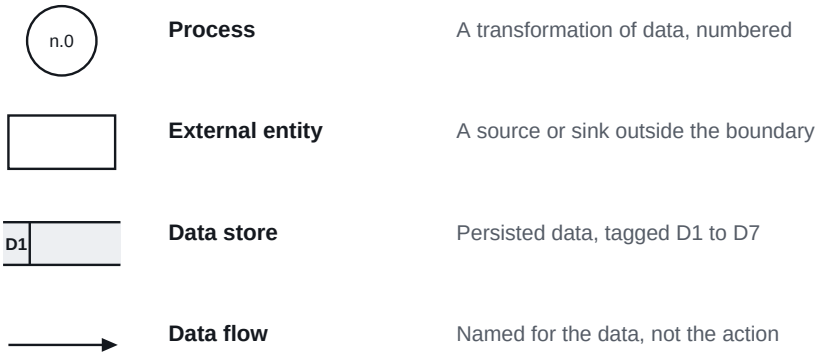



*Figure 4.1 Notation used in the data flow diagrams*

## 4.2 Level 0: Context Diagram

Figure 4.2 fixes the system boundary. Aidly appears as a single process numbered zero. Four human external entities interact with it, and ERSS 112 appears as a fifth external entity that receives an escalation handoff and supplies no data in return. Placing 112 outside the boundary is the diagrammatic statement of the commitment made in Chapter 1.

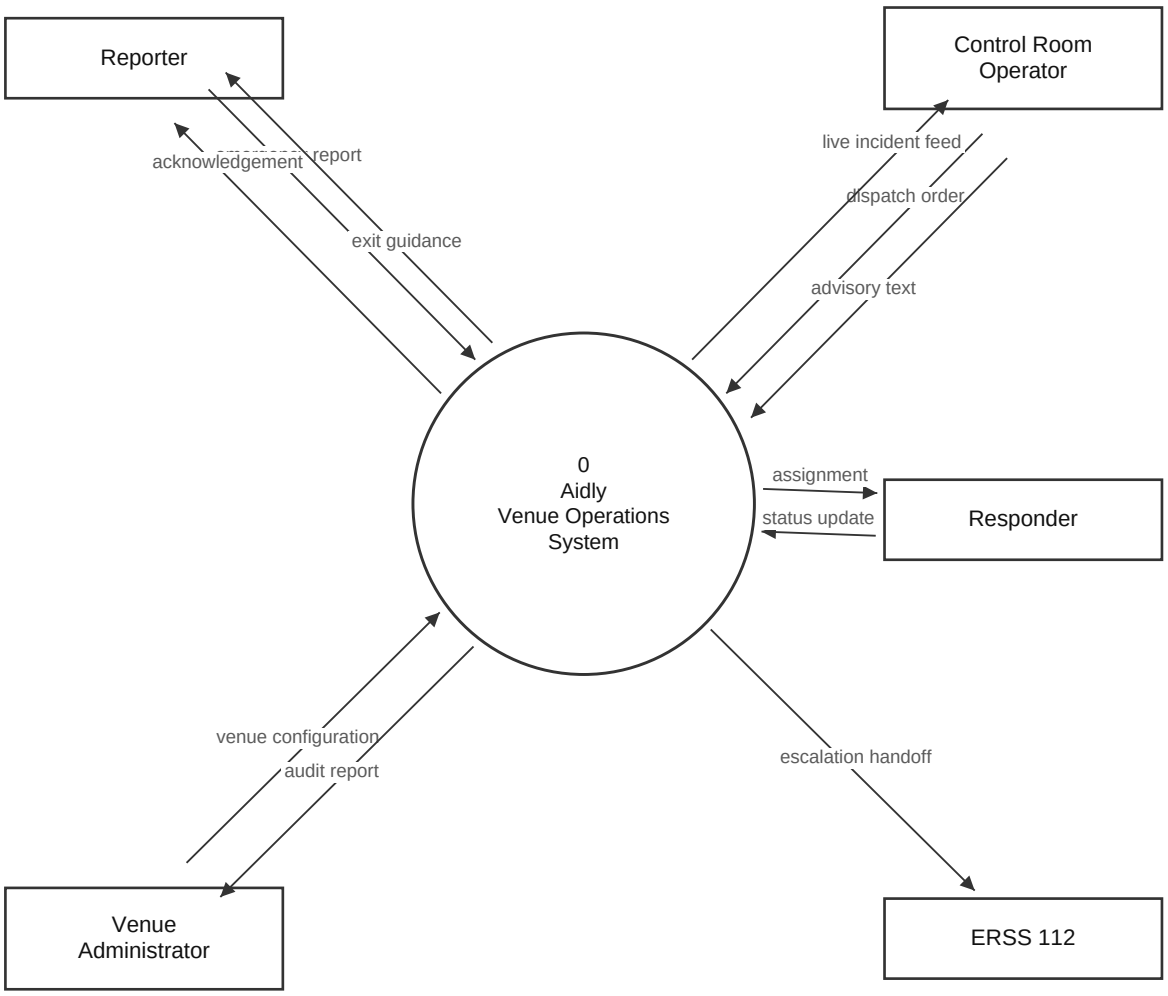



*Figure 4.2 Context diagram, level 0*

## Flows crossing the boundary

These eleven flows are the complete interface between the system and everything outside it.

| From | To | Data flow |
|---|---|---|
| Reporter | System | emergency report |
| System | Reporter | acknowledgement |
| System | Reporter | exit guidance |
| Control Room Operator | System | dispatch order |
| Control Room Operator | System | advisory text |
| System | Control Room Operator | live incident feed |
| System | Responder | assignment |
| Responder | System | status update |
| Venue Administrator | System | venue configuration |
| System | Venue Administrator | audit report |
| System | ERSS 112 | escalation handoff |

*Table 4.1 The eleven flows that cross the system boundary*

## 4.3 Level 1 Data Flow Diagram

Figure 4.3 decomposes process zero into eight processes. Stores D1 to D7 are the principal logical stores of the operational system, and each corresponds to a table of the physical schema. The authentication store D8 is deliberately outside this diagram and is documented in Section 4.7 and implemented in Chapter 7. The figure is set sideways because a diagram of this size is only legible across the long edge of the page.

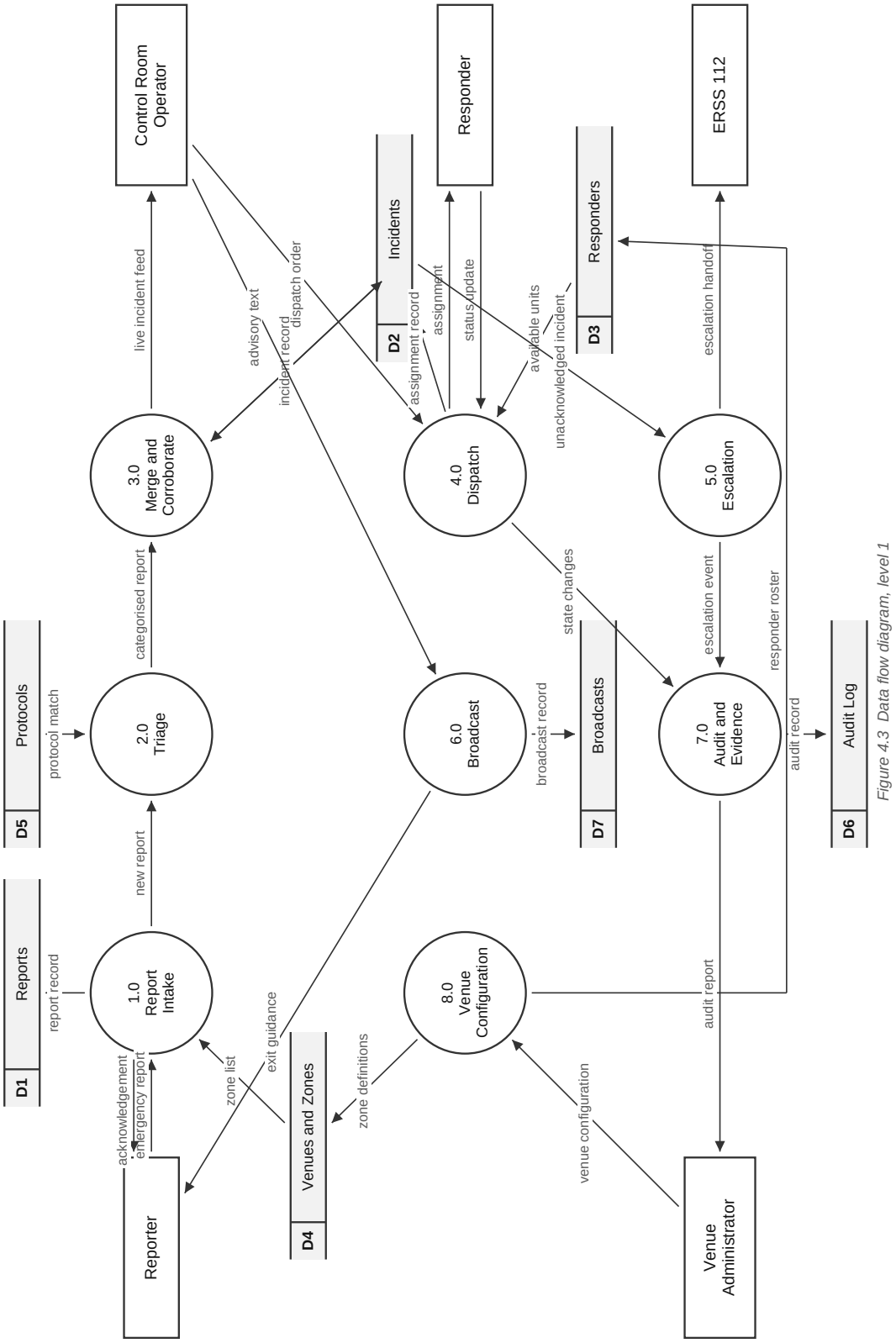



*Figure 4.3 Data flow diagram, level 1*

## 4.4 Reading the Level 1 Diagram

Three conventions in the diagram carry meaning that is easy to miss.

- **Store arrows show direction.** An arrow pointing into a store is a write and an arrow pointing out of it is a read. Process 2.0 reads the protocol library and never modifies it. Process 4.0 reads

the responder roster to choose a unit. Process 8.0 is the only process that writes the venue configuration.

- **The diagram is not a sequence.** Only 1.0, 2.0 and 3.0 form a chain, because a report genuinely passes through those three in order. Dispatch and broadcast are started by the operator, and escalation is started by a timer running over the incident store, so none of them is reached by completing the process above it.
- **Every flow is named for data, not for an action.** A flow is called *categorised report* rather than *categorise the report*. This is what separates a data flow diagram from a flow chart.

The diagram is also **balanced** against the context diagram. Every flow that crosses the boundary in figure 4.2 appears in figure 4.1 with the same name, and no flow appears in one and not the other. Table 4.1 lists all eleven. Balancing is what allows the two levels to be read as one model rather than two drawings of the same idea.

Process 8.0 exists for that reason. The context diagram shows the venue administrator supplying a venue configuration, so some process at level 1 has to receive it. Without 8.0 the administrator would send data into a system that had nowhere to put it, and the zone list that 1.0 depends on would arrive from nowhere.

| Process | Reads | Writes | Principal output |
|---|---|---|---|
| 1.0 Report Intake | D4 | D1 | report, acknowledgement |
| 2.0 Triage | D5 | D1 | categorised report |
| 3.0 Merge and Corroborate | D2 | D2 | incident, live feed |
| 4.0 Dispatch | D3 | D2 | assignment |
| 5.0 Escalation | D2 | D6 | escalation handoff |
| 6.0 Broadcast | D4 | D7 | exit guidance |
| 7.0 Audit and Evidence | D2 | D6 | audit report |
| 8.0 Venue Configuration | None | D3, D4 | zone and roster definitions |

*Table 4.2 Store access by process*

## 4.5 Process Specifications

Each level 1 process is specified by what it receives, what it does, and what it produces. These specifications are the bridge between the diagram and the code, and they are what the later increments are built against.

### 1.0 Report Intake

| | |
|---|---|
| Input | emergency report from the Reporter |
| Processing | Validates the submitted fields. Resolves the device position to the nearest zone, or accepts the gate the reporter selected. Stores any photograph and keeps a reference. Honours the discreet flag by suppressing every confirmation signal. |
| Output | report written to D1, passed to 2.0 |

## 2.0 Triage

| | |
|---|---|
| Input | report from 1.0, protocol library from D5 |
| Processing | Assigns a severity tier. Records which method produced the tier, so an operator can tell a rule from a model from a human judgement. Leaves the probability empty when no trained classifier has scored the report. |
| Output | categorised report passed to 3.0 |

## 3.0 Merge and Corroborate

| | |
|---|---|
| Input | categorised report from 2.0, open incidents from D1 |
| Processing | Searches for an open incident of the same category, in the same zone, within the configured time window. Attaches the report to that incident and increments the corroboration count, or opens a new incident. |
| Output | incident written to D2, passed to 4.0 |

## 4.0 Dispatch

| | |
|---|---|
| Input | incident from 3.0, roster from D3 |
| Processing | Derives the required role from the category, adds any required attribute, and orders available responders by distance from the incident zone. Applies the documented fallback when nothing matches. |
| Output | assignment to the Responder, status written to D2 |

## 5.0 Escalation

| | |
|---|---|
| Input | assignment state from 4.0, incident from D2, venue detail from D4 |
| Processing | Runs a timer against every high severity incident. Where an assignment is not acknowledged inside the configured interval, raises the incident and prepares the handoff payload for the control room to send. |
| Output | escalation handoff to ERSS 112, event written to D6 |

## 6.0 Broadcast

| | |
|---|---|
| Input | operator instruction, venue and zone detail from D4 |
| Processing | Composes an advisory addressed either to the whole venue or to named zones, and delivers it to every connected reporting client. |
| Output | advisory to attendees, record written to D7 |

## 8.0 Venue Configuration

| | |
|---|---|
| Input | venue configuration from the Venue Administrator |
| Processing | Records the venue, its named zones and their positions, and the responder roster with each unit's role and attributes. This is the only process that writes either store, so the data every other process depends on has one owner. |
| Output | zone definitions written to D4, responder roster written to D3 |

### 7.0 Audit and Evidence

| | |
|---|---|
| Input | state changes from every other process, incident from D2 |
| Processing | Appends an immutable event for every state change and for every access to attached evidence. Never updates or deletes an existing row. |
| Output | audit report to the Venue Administrator, events written to D6 |

## 4.6 Data Dictionary: Data Flows

The notation is standard. = is composed of, + is and, **{ }** encloses a repeating group, **( )** encloses an optional item, and **[ | ]** encloses a choice of exactly one alternative.

| Name | Definition |
|---|---|
| emergency report | category + zone reference + timestamp + ( description ) + ( photograph reference ) + ( callback number ) + discreet flag |
| categorised report | emergency report + severity tier + triage method |
| incident | incident identifier + category + zone reference + severity tier + corroboration count + status + { report identifier } |
| assignment | incident identifier + responder identifier + issued at + acknowledgement deadline |
| assignment acknowledgement | assignment identifier + responder identifier + acknowledged at |
| status update | incident identifier + [ en route \| on scene \| resolved ] + recorded at |
| unacknowledged P1 | incident identifier + severity tier + elapsed seconds |
| escalation handoff | incident identifier + venue reference + zone name + category + severity tier + summary text |
| advisory | venue reference + ( zone reference ) + message text + issued at |
| live incident feed | { incident + assignment status } |
| venue setup | venue name + { zone name + zone position + exit flag } + { responder record } |
| audit report | { incident identifier + { event type + actor + occurred at } } |
| venue configuration | venue name + { zone name + zone position + exit flag } + { responder code + name + role + attributes } |
| zone definitions | { zone identifier + zone name + zone position + exit flag } |
| responder roster | { responder identifier + code + name + role + attributes + venue reference } |
| acknowledgement | incident identifier + reference + status + ( zone name ) |
| exit guidance | venue reference + ( zone reference ) + message text |
| advisory text | message text + ( zone reference ) + issued by |
| credentials | email address + password |
| access token | account identifier + role + ( venue reference ) + issued at + expires at + signature |

*Table 4.3 Data flow definitions*

## 4.7 Data Dictionary: Data Stores

Each store named in the two diagrams is defined below, in the same notation.

| Tag | Store | Definition |
|---|---|---|
| D1 | Reports | report identifier + category + zone reference + ( description ) + ( photograph reference ) + ( callback number ) + discreet flag + received at + ( incident identifier ) |
| D2 | Incidents | incident identifier + venue reference + zone reference + category + severity tier + ( severity probability ) + triage method + corroboration count + status + opened at + ( resolved at ) |
| D3 | Responders | responder identifier + venue reference + code + name + role + position + availability flag + last seen at |
| D4 | Venues and Zones | venue identifier + name + slug + centre position + { zone identifier + zone name + zone position + exit flag } |
| D5 | Protocols | protocol identifier + category + title + body text + embedding vector |
| D6 | Audit Log | event identifier + incident identifier + event type + actor reference + occurred at + payload |
| D7 | Broadcasts | broadcast identifier + venue reference + ( zone reference ) + message text + issued by + issued at |
| D8 | Users | account identifier + email address + password hash + display name + role + ( venue reference ) + active flag + created at |

*Table 4.4 Data store definitions*

## 4.8 Data Dictionary: Data Elements

The elements below appear inside several of the flows and stores already defined, so they are given once here rather than repeated.

The separation of **severity probability** from **corroboration count** is deliberate. The first is a claim a model makes about one report. The second is evidence of how many people independently reported the same thing. Combining them would let a weak model appear as corroborated fact, and an operator could act on it. The probability stays null until a trained classifier exists: an absent number is honest, an invented one is not.

| Element | Type and permitted values |
|---|---|
| category | [ fire \| medical \| crowd \| accident \| violence \| harassment \| other ] |
| severity tier | [ P1 \| P2 \| P3 \| P4 ], where P1 is the most urgent |
| severity probability | Real number between 0 and 1. Null until a trained classifier produces it. |
| triage method | [ rule \| classifier \| operator ] |
| status | [ triaging \| verified \| dispatched \| en route \| on scene \| resolved ] |
| role | [ medic \| fire \| marshal \| security ] |
| user role | [ operator \| responder \| admin ] |
| discreet flag | Boolean. When true the client suppresses all sound, vibration and visible state change. |
| corroboration count | Integer, at least 1. Counts independent reports merged into one incident. |
| position | Geographic point, latitude and longitude, SRID 4326 |
| password hash | 60 character bcrypt digest. The salt is carried inside the digest, so no separate salt element exists. |
| email address | Unique across the users store, stored trimmed and lowered |

*Table 4.5 Data element definitions*

# 5
# Use Case Diagram

## 5.1 Actors

Five actors interact with the system. Four are people and one is an external service.

| Actor | Description |
|---|---|
| Reporter | Any attendee inside the venue. Not registered and not authenticated. Interacts only with the reporting path. |
| Responder | A marshal, medic, fire warden or security member on duty. Authenticated. Receives and acts on assignments. |
| Control Room Operator | Supervises the venue. Authenticated. Sees every incident, dispatches responders, issues broadcasts and initiates escalation. |
| Venue Administrator | Configures the venue, its zones and its responder roster. Authenticated. |
| ERSS 112 | External system actor. Receives an escalation handoff. Supplies no data to Aidly. |

*Table 5.1 Actors of the system*

Only the Reporter is unauthenticated, and that is a requirement rather than an omission. An attendee in an emergency cannot be asked to create an account. Every other actor operates the venue and is therefore subject to FR 15.

## 5.2 The Diagram

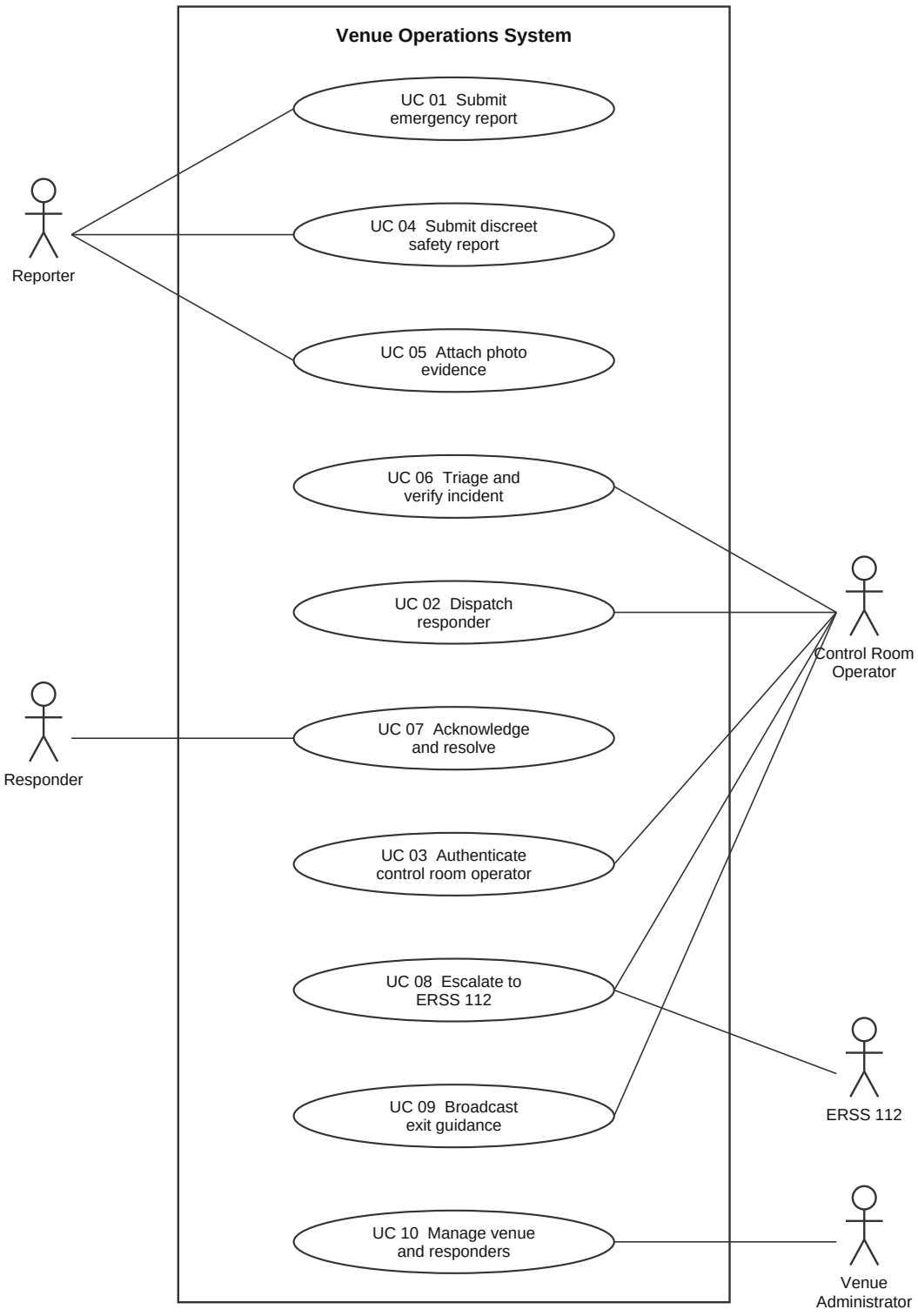



*Figure 5.1 Use case diagram*

## 5.3 Use Case Inventory

The ten use cases below are the complete inventory for the system.

| ID | Use case | Primary actor | Requirements |
|---|---|---|---|
| UC 01 | Submit emergency report | Reporter | FR 01 to FR 04, FR 06 to FR 08 |
| UC 02 | Dispatch responder | Control Room Operator | FR 11 |
| UC 03 | Authenticate control room operator | Control Room Operator | FR 15 |
| UC 04 | Submit discreet safety report | Reporter | FR 14 |
| UC 05 | Attach photo evidence | Reporter | FR 05 |
| UC 06 | Triage and verify incident | Control Room Operator | FR 09, FR 10 |
| UC 07 | Acknowledge and resolve assignment | Responder | FR 12 |
| UC 08 | Escalate to ERSS 112 | Control Room Operator | FR 13, FR 17 |
| UC 09 | Broadcast exit guidance | Control Room Operator | FR 16 |
| UC 10 | Manage venue and responders | Venue Administrator | FR 18 |

*Table 5.2 Use case inventory. Templates for the three core cases follow in Chapter 6.*

# 6
# Use Case Templates

Templates are given for the three core requirements of the application: the reporter path that begins every incident, the operator path that resolves it, and the authentication that guards both. The third is implemented and demonstrated in Chapter 7.

## 6.1 UC 01: Submit Emergency Report

This is the path that begins every incident, and the only one open to someone who is not authenticated.

| | |
|---|---|
| **Use case ID** | UC 01 |
| **Name** | Submit emergency report |
| **Primary actor** | Reporter, an attendee inside the venue |
| **Requirements** | FR 01, FR 02, FR 03, FR 04, FR 05, FR 06, FR 07, FR 08 |
| **Description** | An attendee who witnesses an emergency sends a categorised report to the venue control room in a single action, without registering. |
| **Preconditions** | The reporter has opened the reporting screen on a device with a network connection. The venue is configured with at least one zone. |
| **Postconditions** | A report is persisted in D1 and resolved to a zone. It either opens a new incident in D2 or increments the corroboration count of an existing one. The reporter holds an incident identifier. |
| **Trigger** | The reporter activates the reporting control on the landing screen. |
| **Main flow** | 1. The reporter activates the reporting control.<br>2. The system presents the category chooser.<br>3. The reporter selects a category.<br>4. The system requests the device position.<br>5. The system resolves the position to the nearest zone and displays it.<br>6. The reporter optionally enters a description of at most 150 characters.<br>7. The reporter optionally attaches a photograph, which is uploaded and referenced against the report.<br>8. The reporter optionally enters a callback number.<br>9. The reporter submits the report.<br>10. The system reports each stage of submission as it completes.<br>11. The system persists the report, passes it to triage, issues an incident identifier and shows a live status. |
| **Alternate A1** | **Position unavailable.** At step 4 the browser refuses or fails to supply a position. The system displays a notice and presents the list of gates. The reporter selects the nearest gate and the flow resumes at step 6. The report is never blocked. |
| **Alternate A2** | **Duplicate event.** At step 11 an open incident exists with the same category and zone inside the configured time window. The system attaches the report to that incident and increments its corroboration count instead of opening a new one. |
| **Alternate A3** | **Discreet submission.** The category is a personal safety category and the reporter selects discreet mode. The system suppresses all sound, vibration and visible state change, and sends no notification back to that device. |

| | |
|---|---|
| **Exception E1** | **Submission fails.** The request does not complete. The system retains the entered data, states that the report has not been sent, and offers a retry. Silent failure is not permitted. |
| **Exception E2** | **Photograph rejected.** The upload fails or the file is not an image. The system reports it and allows the report to be submitted without the photograph, because FR 05 is optional and NFR 03 forbids an optional input from blocking submission. |
| **Exception E3** | **No zone configured.** The system accepts the report against the venue centre and flags it for operator attention. |
| **Frequency** | High. Expected to be the most invoked use case in the system. |

## 6.2 UC 02: Dispatch Responder

This is the operator path that turns a verified incident into a responder on the ground.

| | |
|---|---|
| **Use case ID** | UC 02 |
| **Name** | Dispatch responder |
| **Primary actor** | Control Room Operator |
| **Secondary actor** | Responder |
| **Requirements** | FR 11, FR 12 |
| **Description** | The operator assigns the most suitable available responder to an open incident and tracks the assignment to resolution. |
| **Preconditions** | The operator is authenticated as in UC 03. An incident exists in D2 with status triaging or verified. At least one responder is recorded in D3. |
| **Postconditions** | The incident status becomes dispatched, an assignment is written, the responder availability flag is cleared, and the action is recorded in D6. |
| **Trigger** | The operator selects an open incident and requests dispatch. |
| **Main flow** | 1. The operator selects an open incident from the live feed.<br>2. The system displays its category, zone, severity tier and corroboration count.<br>3. The operator requests a dispatch recommendation.<br>4. The system determines the required role from the incident category.<br>5. The system queries D3 for available responders holding that role, ordered by distance from the incident zone.<br>6. The system presents the nearest matching responder.<br>7. The operator confirms the assignment.<br>8. The system records it, marks the responder unavailable and notifies the responder device.<br>9. The responder acknowledges and the incident status advances. |
| **Alternate A1** | **Attribute constrained request.** The category is harassment, or an escort has been requested. At step 4 the system also requires the responder attribute recorded for that purpose, and the ordering at step 5 is restricted accordingly. |
| **Alternate A2** | **Operator override.** At step 7 the operator rejects the recommendation and selects a different responder. The system records both the recommendation and the override in D6. |
| **Exception E1** | **No matching responder.** The query at step 5 returns nothing. The system does not fail silently. It widens the search to any available responder, marks the assignment as a fallback, and raises the incident for operator attention. |
| **Exception E2** | **Not acknowledged.** The responder does not acknowledge within the configured interval. Escalation is triggered, as in FR 12. |
| **Frequency** | Moderate. Once per incident, occasionally more where an override or a reassignment occurs. |

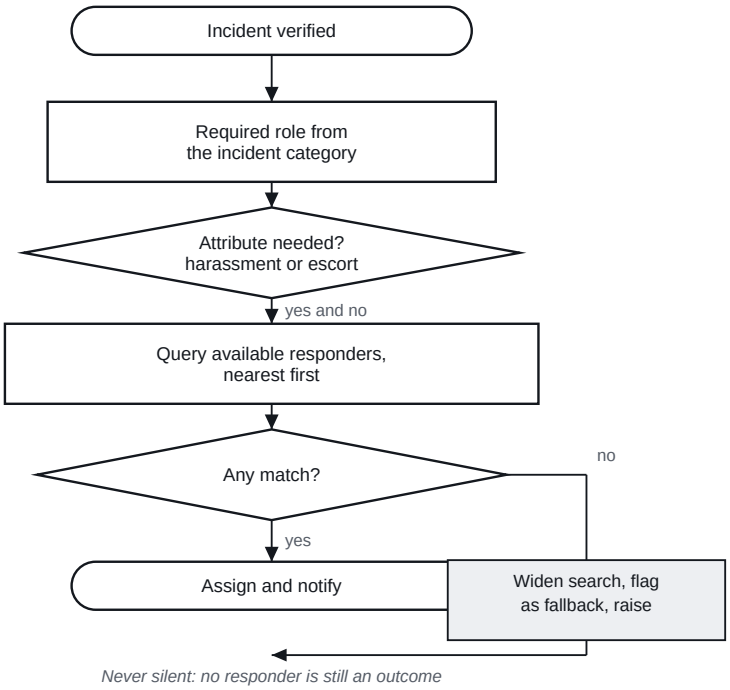



*Figure 6.1 Responder matching decision flow*

## 6.3 UC 03: Authenticate Control Room Operator

This case guards every operational screen. It is implemented and demonstrated in Chapter 7.

| | |
|---|---|
| **Use case ID** | UC 03 |
| **Name** | Authenticate control room operator |
| **Primary actor** | Control Room Operator |
| **Requirements** | FR 15, NFR 05, NFR 06, NFR 07 |
| **Description** | An operator proves identity to the system and receives a time limited access token that authorises subsequent operational requests. |
| **Preconditions** | An account exists in D8 with the active flag set and a stored password hash. The application can reach the database. |
| **Postconditions** | On success the operator holds a signed token carrying the account identifier, the role and the venue reference. On failure no token is issued and nothing is disclosed about the account. |
| **Trigger** | The operator submits credentials on the sign in screen. |
| **Main flow** | 1. The operator supplies an email address and a password.<br>2. The system normalises the address and looks up the account in D8.<br>3. The system verifies the password against the stored hash.<br>4. The system confirms the account is active.<br>5. The system issues a signed token carrying the identifier, role and venue reference.<br>6. The system returns the token, its lifetime and the safe account fields.<br>7. The client sends the token on every later request, and the system reloads the account to confirm it is still active. |
| **Alternate A1** | **Unknown account.** At step 2 no account matches. The system verifies the password against a fixed dummy hash so the elapsed time does not differ, then follows the failure path. |

| | |
|---|---|
| **Exception E1** | **Wrong password.** Verification at step 3 fails. The system returns status 401 with the message *Incorrect email or password*. The same message is used for every failure cause, so the response cannot be used to discover which accounts exist. |
| **Exception E2** | **Deactivated account.** The check at step 4 fails. The system responds exactly as for a wrong password. |
| **Exception E3** | **Malformed stored hash.** Verification raises rather than returning. The system treats the result as a failure and continues, so one corrupt row cannot disable the sign in endpoint. |
| **Exception E4** | **Database unreachable.** The system returns a server error and issues no token. It never falls back to a default or shared credential. |
| **Frequency** | Low. Once per operator per shift, plus token renewals. |

## 6.4 Incident State Transitions

The three templates above move an incident between states. The complete set of permitted transitions is given below. Any transition not listed is rejected, which is what prevents an incident from being resolved before it was ever dispatched.

| From | Event | To | Recorded by |
|---|---|---|---|
| triaging | Severity assigned and operator confirms | verified | 2.0 |
| triaging | Merged into an existing incident | closed as duplicate | 3.0 |
| verified | Operator confirms an assignment | dispatched | 4.0 |
| dispatched | Responder acknowledges | en route | 4.0 |
| dispatched | Acknowledgement interval elapses | verified, and raised | 5.0 |
| en route | Responder reports arrival | on scene | 4.0 |
| on scene | Responder reports completion | resolved | 4.0 |
| any state | Operator initiates escalation | escalated | 5.0 |

*Table 6.1 Permitted incident state transitions*

The transition from dispatched back to verified matters most. An assignment that is never acknowledged must return the incident to the queue rather than leave it marked as handled, because an incident believed to be handled is one nobody looks at again.

# 7
# Login Module Connection with Database

## 7.1 What Was Built

This chapter documents Increment I of the plan in Table 3.2: a sign in module connected to a live PostgreSQL database. It implements UC 03 and satisfies FR 15, NFR 05, NFR 06 and NFR 07. The module is written in Python using the FastAPI framework and SQLAlchemy, and every listing here is the code that produced the results in Section 7.12.

| Component | Choice | Reason |
|---|---|---|
| Language | Python 3.12 | Matches the rest of the project and the tooling already in use. |
| Web framework | FastAPI | Validates request bodies against declared schemas before any handler runs. |
| Database | PostgreSQL 18 | Relational storage with the geospatial and vector extensions the later increments require. |
| Object mapping | SQLAlchemy 2.0 | Parameterised queries by construction, which removes injection as a class of defect. |
| Password hashing | bcrypt | Per password salt and a deliberately expensive derivation. |
| Token format | Signed JSON Web Token | Carries the role so routine checks need no database round trip. |

*Table 7.1 Implementation choices*

## 7.2 Structure of the Module

The module is split across four files. Hashing and token handling are isolated from the database, the database work is isolated from the HTTP layer, and the HTTP layer contains no security logic of its own. The hashing scheme can therefore be replaced without touching any endpoint, which is exactly what happened during construction and is described in Section 7.12.

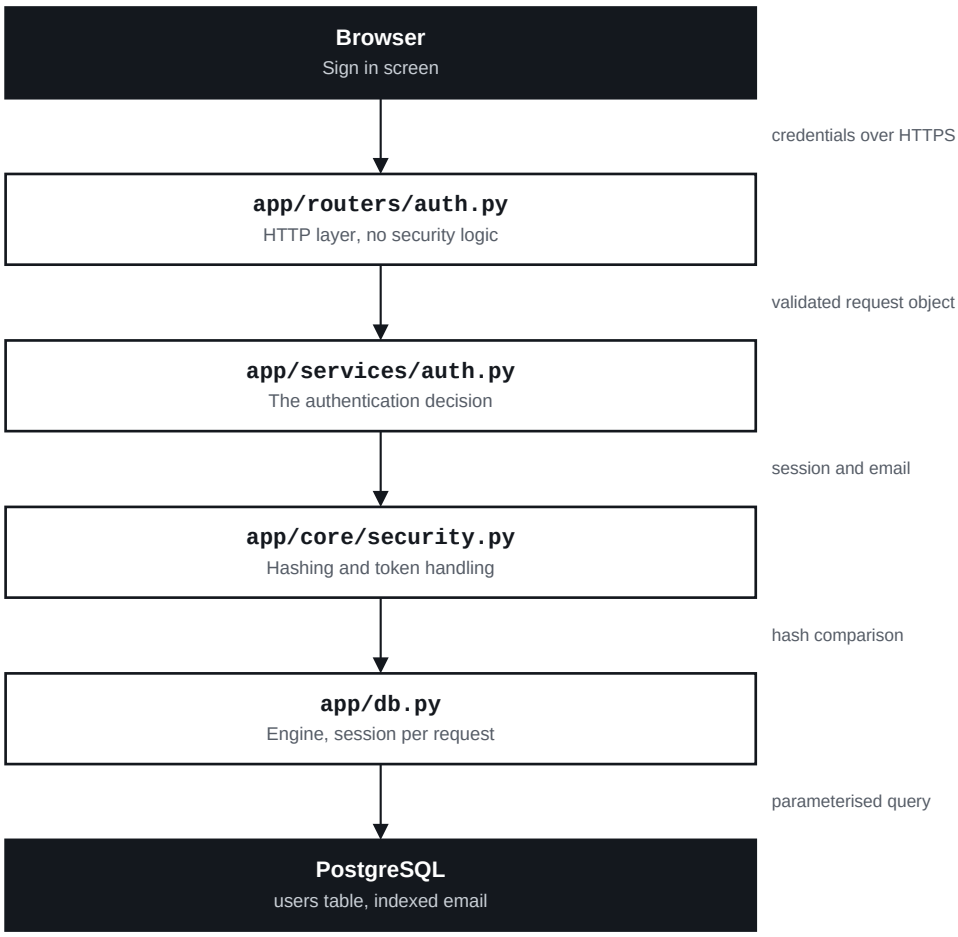



*Figure 7.1 Structure of the login module*

## 7.3 The Users Table

The users table is store D8 of the data dictionary. It replaces the single shared passcode the prototype used, so that each account carries its own hash, its own role and its own active flag.

app/models.py

```python
class User(Base, PkMixin):
    """One account. Replaces the shared passcode the prototype used."""
    __tablename__ = "users"

    email: Mapped[str] = mapped_column(String(255), unique=True, index=True)
    password_hash: Mapped[str] = mapped_column(String(255))
    display_name: Mapped[str] = mapped_column(String(120))
    role: Mapped[UserRole] = mapped_column(Enum(UserRole, name="user_role"))
    venue_id: Mapped[uuid.UUID | None] = mapped_column(
        ForeignKey("venues.id", ondelete="SET NULL"))
    is_active: Mapped[bool] = mapped_column(Boolean, default=True)
```

*Listing 7.1 Definition of the users table*

| Column | Type | Notes |
|---|---|---|
| id | UUID | Primary key. Not sequential, so account count is not disclosed. |
| email | String(255) | Unique and indexed. The lookup in Section 7.7 is an index seek. |
| password_hash | String(255) | bcrypt digest. No separate salt column is needed. |
| display_name | String(120) | Shown to the operator after sign in. |

| Column | Type | Notes |
|---|---|---|
| role | Enum | operator, responder or admin. Travels inside the token. |
| venue_id | UUID, nullable | Scopes the account to one venue. |
| is_active | Boolean | Checked on sign in and again on every request. |

*Table 7.2 Columns of the users table*

## 7.4 The Connection Layer

A single engine holds a pooled set of connections for the life of the process. Each request receives its own session, committed if the request succeeds and rolled back if it raises, and closed in either case.

app/db.py

```
engine = create_engine(
    str(settings.database_url),
    pool_pre_ping=True,      # drop connections the database already closed
    pool_size=10,
    max_overflow=20,
    echo=settings.debug,
)

SessionLocal = sessionmaker(bind=engine, autoflush=False, expire_on_commit=False)


def get_db() -> Generator[Session, None, None]:
    """Commits on success, rolls back on any exception, always closes."""
    db = SessionLocal()
    try:
        yield db
        db.commit()
    except Exception:
        db.rollback()
        raise
    finally:
        db.close()
```

*Listing 7.2  Engine, session factory and the request scoped session*

The connection string and the token signing secret are read from the environment and have no working defaults. If either is missing the application refuses to start rather than falling back to an insecure value, which is the failure the prototype exhibited with its shared operator passcode.

## 7.5 Password Hashing

Passwords are stored only as bcrypt hashes. bcrypt generates a random salt for every password and applies a deliberately expensive derivation, so two identical passwords produce different stored values and a stolen table cannot be reversed by lookup.

app/core/security.py

```python
BCRYPT_MAX_BYTES = 72


def _prepare(plain: str) -> bytes:
    """Encode and truncate on a character boundary, never mid codepoint."""
    raw = plain.encode("utf-8")
    if len(raw) <= BCRYPT_MAX_BYTES:
        return raw
    return raw[:BCRYPT_MAX_BYTES].decode("utf-8", "ignore").encode("utf-8")


def hash_password(plain: str) -> str:
    return bcrypt.hashpw(_prepare(plain), bcrypt.gensalt()).decode("ascii")


def verify_password(plain: str, hashed: str) -> bool:
    """False on a malformed stored hash rather than raising, so one corrupt
    row cannot break the sign in endpoint."""
    try:
        return bcrypt.checkpw(_prepare(plain), hashed.encode("ascii"))
    except (ValueError, TypeError):
        return False
```

*Listing 7.3  Hashing and verification*

The 72 byte constant is not arbitrary. bcrypt ignores everything past the 72nd byte of a password. Left implicit, two different long passwords could silently share a hash, so the limit is applied in one visible place and the truncation respects character boundaries rather than cutting a multibyte character in half.

## 7.6 Access Tokens

app/core/security.py

```python
def create_access_token(subject: str, role: str,
                        venue_id: str | None = None) -> tuple[str, int]:
    """Returns the encoded token and its lifetime in seconds."""
    expires_in = settings.access_token_minutes * 60
    now = datetime.now(timezone.utc)
    payload = {
        "sub": subject,
        "role": role,
        "venue_id": venue_id,
        "iat": now,
        "exp": now + timedelta(seconds=expires_in),
    }
    token = jwt.encode(payload, settings.jwt_secret,
                       algorithm=settings.jwt_algorithm)
    return token, expires_in
```

*Listing 7.4  Issuing a signed access token*

The role travels inside the token so that routine authorisation checks need no database round trip. Anything that must be current, such as whether the account is still active, is read from the database instead. That division is what allows NFR 07 to be satisfied without making every request expensive.

## 7.7 The Authentication Service

The service is the only component that decides whether a credential is acceptable. Three rules govern it, and each answers a specific attack.

app/services/auth.py

```python
_DUMMY_HASH = "$2b$12$C6UzMDM.H6dfI/f/IKcEe.rJXQnGDDPFHOCkCcTLB0P9C7dQKrHQO"


def authenticate(db: Session, email: str, password: str) -> User | None:
    """Returns the user on success, None on every kind of failure."""
    normalised = email.strip().lower()
    user = db.execute(
        select(User).where(User.email == normalised)
    ).scalar_one_or_none()

    if user is None:
        # Verified anyway so the elapsed time does not reveal the answer.
        verify_password(password, _DUMMY_HASH)
        return None

    if not verify_password(password, user.password_hash):
        return None

    if not user.is_active:
        return None

    return user
```

*Listing 7.5 The authentication decision*

- **One outcome for every failure.** The function returns None whether the account is unknown, the password is wrong, or the account is deactivated. The caller therefore cannot produce different responses for different causes.
- **Constant work when the account is unknown.** A dummy hash is verified even though no user was found. Without this the unknown account path would return measurably faster, and an attacker could enumerate valid addresses by timing alone. This is what satisfies NFR 06.
- **Normalisation before lookup.** The address is trimmed and lowered, so stored addresses match consistently and case cannot be used to create a duplicate account.

## 7.8 The Endpoints

app/routers/auth.py

```python
INVALID = HTTPException(
    status_code=status.HTTP_401_UNAUTHORIZED,
    detail="Incorrect email or password",
    headers={"WWW-Authenticate": "Bearer"},
)


@router.post("/login", response_model=TokenResponse)
def login(payload: LoginRequest, db: Session = Depends(get_db)) -> TokenResponse:
    user = authenticate(db, payload.email, payload.password)
    if user is None:
        raise INVALID

    token, expires_in = create_access_token(
        subject=str(user.id),
        role=user.role.value,
        venue_id=str(user.venue_id) if user.venue_id else None,
    )
    return TokenResponse(access_token=token, expires_in=expires_in,
                         user=UserOut.model_validate(user))
```

*Listing 7.6 The sign in endpoint*

app/routers/auth.py

```python
def current_user(
    creds: HTTPAuthorizationCredentials | None = Depends(bearer),
    db: Session = Depends(get_db),
) -> User:
    """Dependency for every protected route. The account is reloaded rather
    than trusted from the token, so deactivating a user takes effect at once."""
    if creds is None:
        raise INVALID

    claims = decode_access_token(creds.credentials)
    if claims is None or "sub" not in claims:
        raise INVALID

    user = db.get(User, claims["sub"])
    if user is None or not user.is_active:
        raise INVALID

    return user
```

*Listing 7.7 The authorisation dependency used by protected routes*

The response is shaped by a schema that has no password field of any kind, so the stored hash cannot be returned by accident. This is a stronger guarantee than removing the field before sending, because there is no code path in which it could be present.

## 7.9 The Interface Offered by the Module

The module exposes two endpoints and one page. The contract below is what any client, including the screen in Section 7.10, must satisfy.

| Method and path | Request | Success | Failure |
|---|---|---|---|
| POST /api/auth/login | email address and password, as JSON | 200 with token, token type, lifetime and the safe account fields | 401 for any bad credential. 422 if the body fails validation. |
| GET /api/auth/me | Bearer token in the Authorization header | 200 with the safe account fields | 401 for a missing, malformed, expired or tampered token, and for a deactivated account. |
| GET /login | None | 200 with the sign in page | Not applicable |

*Table 7.3 Interface contract of the login module*

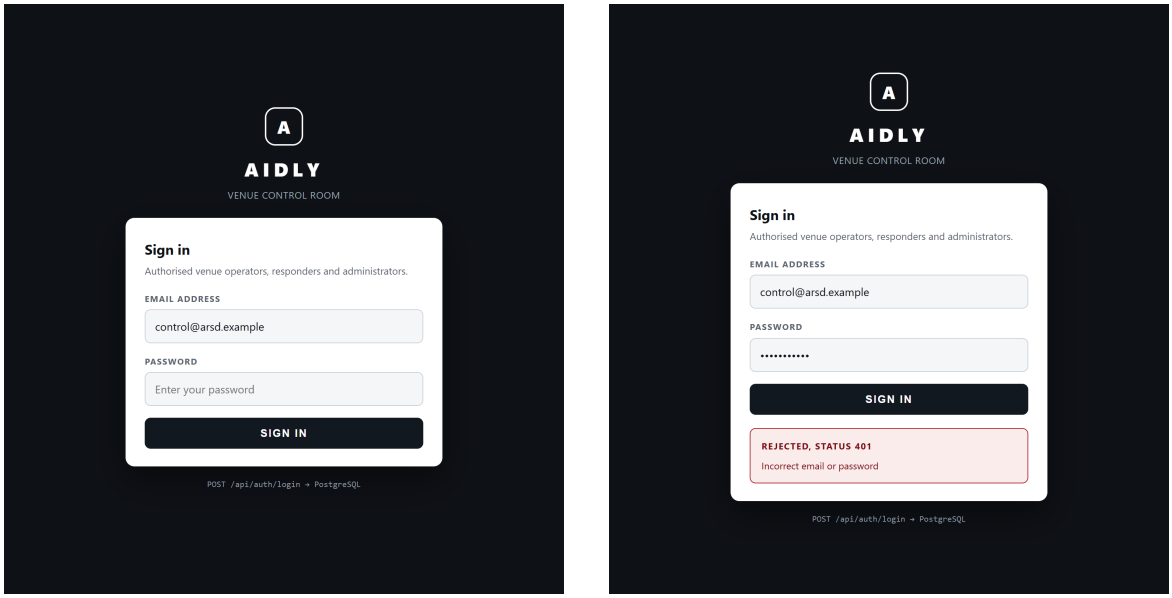
Validation failures return 422 rather than 401, and that distinction is deliberate. A malformed request body is a programming error in the client, not a failed sign in attempt, and conflating the two would hide real integration faults behind an authentication message. No detail about the account is disclosed in either case.

| Status | Meaning | Body |
|---|---|---|
| 200 | Authenticated | Token, token type, lifetime, account fields |
| 401 | Rejected | Incorrect email or password |
| 422 | Request body invalid | Which field failed and why |
| 500 | Database unreachable | No token, no account detail |

*Table 7.4 Status codes returned by the module*

## 7.10 The Sign In Screen

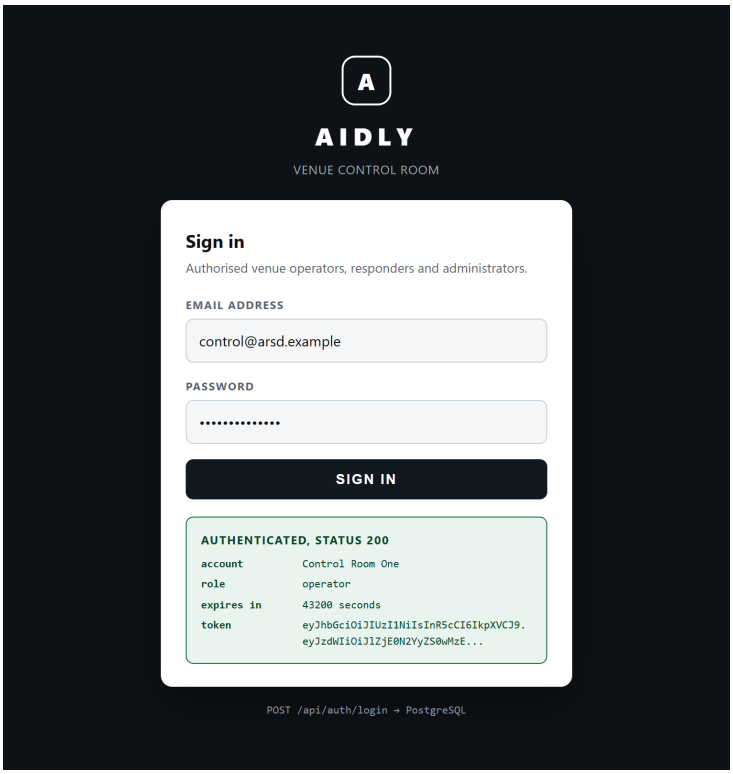
The screen is served by the same application that owns the endpoint, so the module can be demonstrated end to end without a separate client. The three states below are captured from the running system.




Awaiting credentials

Rejected, status 401

*Figure 7.3 The sign in screen, and the response to a wrong password. The message names neither the account nor the reason.*




Authenticated, status 200

*Figure 7.4 A successful sign in. The token, the role and the lifetime are returned, and the account is reloaded through the protected route before anything is displayed.*

## 7.11 Sequence of One Authentication

The sequence below traces one successful sign in from the operator through to the database and back.

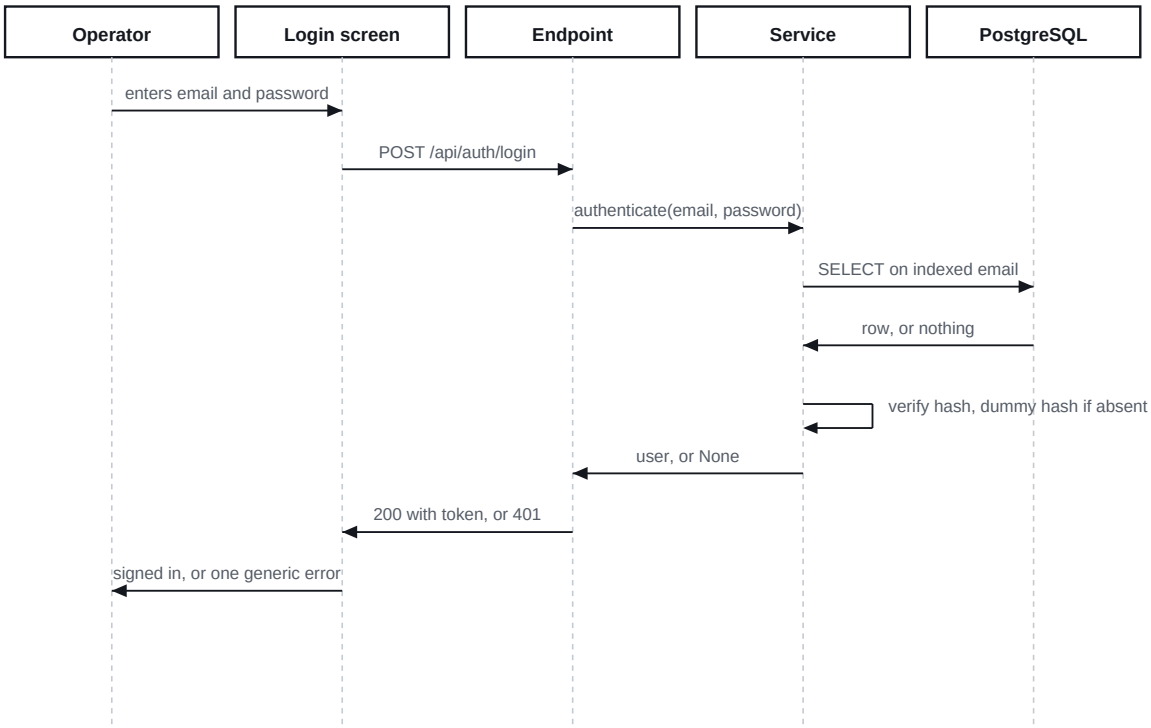



*Figure 7.2  One authentication, end to end*

The self directed step at the service is the timing defence from Section 7.7. It runs whether or not a row came back, which is what makes the two failure paths indistinguishable from outside.

## 7.12 Execution Against the Database

The transcript below is unedited output from running the module against PostgreSQL 18 with the geospatial and vector extensions enabled. The run creates the schema, seeds one operator account, performs a successful sign in, uses the returned token on a protected route, and then exercises each rejection path.

```
execution transcript
```

```
STEP 1  create schema in PostgreSQL
  tables: broadcasts, events, incidents, protocols, reports,
          responders, spatial_ref_sys, users, venues, zones

STEP 2  seed one operator account
  stored id   : ef147f2e-0316-49bd-b7c5-afa30cf4dc88
  stored hash : $2b$12$qtZh22Y0qeBfLtpYMsqT5uR1Zs/Or ...

STEP 3  POST /api/auth/login   correct password
  status: 200
  token : eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJlZjE ...
  type  : bearer  expires_in: 43200
  user  : Control Room One / operator

STEP 4  GET /api/auth/me   with that token
  status: 200  email: control@arsd.example

STEP 5  rejection cases
  wrong password: 401 Incorrect email or password
  unknown email : 401 Incorrect email or password
  no token      : 401
```

*Listing 7.8  Execution transcript*

Ten tables were created, of which the users table is the one this chapter uses. The stored hash begins with the bcrypt version marker, confirming that no plain password reached the database.

The schema actually created in PostgreSQL is reproduced below, taken from the database rather than written by hand. The unique index on the email column is what makes the lookup in Listing 7.5 an index seek and what prevents a second account being created for the same address.

```
generated schema
```

```
CREATE TABLE users (
    email          VARCHAR(255) NOT NULL,
    password_hash  VARCHAR(255) NOT NULL,
    display_name   VARCHAR(120) NOT NULL,
    role           user_role    NOT NULL,
    venue_id       UUID,
    is_active      BOOLEAN      NOT NULL,
    id             UUID         NOT NULL,
    created_at     TIMESTAMP WITH TIME ZONE DEFAULT now() NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY (venue_id) REFERENCES venues (id) ON DELETE SET NULL
);

CREATE UNIQUE INDEX ix_users_email ON users (email);
CREATE INDEX ix_users_created_at ON users (created_at);
```

*Listing 7.9  The users table as created in PostgreSQL*

## 7.13 Verification

Ten test cases were executed against the running module and the live database. Each is stated as an input, an expectation and the observed result, so the run can be repeated and disputed.

| ID | Input | Expected | Observed | Result |
|---|---|---|---|---|
| T01 | Create schema against an empty database | Application tables created | Ten tables created | Pass |

| ID | Input | Expected | Observed | Result |
|---|---|---|---|---|
| T02 | Seed an account with a plain password | Only a hash is stored | Stored value begins $2b$ | Pass |
| T03 | Correct address and password | 200 with a signed token | 200, lifetime 43200 seconds | Pass |
| T04 | That token on a protected route | 200 with the correct account | 200, correct address returned | Pass |
| T05 | Correct address, wrong password | 401 with a generic message | 401 Incorrect email or password | Pass |
| T06 | Address that does not exist | 401, message identical to T05 | 401, identical message | Pass |
| T07 | Protected route with no token | 401 | 401 | Pass |
| T08 | Protected route with an altered token | 401, signature rejected | 401 | Pass |
| T09 | Password longer than 72 bytes | Accepted and verified without error | Hashed and verified | Pass |
| T10 | Verification against a malformed stored hash | Treated as a failure, no exception raised | Returned false | Pass |

*Table 7.5 Test cases executed against the login module*

T05 and T06 are the pair that matters. They confirm that the response to an unknown address is indistinguishable from the response to a wrong password, which is the requirement stated in NFR 06 and the reason the dummy hash in Listing 7.5 exists. T09 and T10 exercise the two defensive paths in Listing 7.3.

**One defect was found and corrected during this increment.** The project originally specified the passlib library for hashing. Current releases of bcrypt no longer expose the version attribute that passlib inspects, and the passlib backend probe raises on the 72 byte limit, so the module failed on its first call. The dependency was replaced with the bcrypt library used directly, which removed a layer rather than adding a workaround. Listing 7.3 shows the corrected code. This is the kind of defect the ordering argument in Section 3.5 was intended to surface early, and it did.

# 8
# References

### Books


1. Pressman, R. S., and Maxim, B. R. (2020). *Software Engineering: A Practitioner's Approach*. 9th edition. McGraw Hill.
2. Aggarwal, K. K., and Singh, Y. (2007). *Software Engineering*. 3rd edition. New Age International Publishers.
3. Jalote, P. (2005). *An Integrated Approach to Software Engineering*. 3rd edition. Narosa Publishing House.


### Documentation and websites consulted

The sources below were consulted during design and implementation.

| Resource | Address | Used for |
|---|---|---|
| FastAPI | fastapi.tiangolo.com | Request validation, dependency injection, and the security utilities used in Listing 7.7 |
| SQLAlchemy 2.0 | docs.sqlalchemy.org | Declarative mapping, typed columns and session handling |
| PostgreSQL 18 | www.postgresql.org/docs | Data types, indexing and extension installation |
| PostGIS | postgis.net/documentation | Geographic point columns and distance ordering for dispatch |
| pgvector | github.com/pgvector/pgvector | Vector column used by the protocol store D5 |
| bcrypt for Python | pypi.org/project/bcrypt | Hashing, salt generation and the 72 byte limit |
| python jose | python-jose.readthedocs.io | Encoding and decoding signed tokens |
| Pydantic | docs.pydantic.dev | Request and response schemas, settings validation |
| Alembic | alembic.sqlalchemy.org | Schema migration approach for Increment I |
| Emergency Response Support System | 112.gov.in | Scope of the national service, used to establish what Aidly must not duplicate |
| National Disaster Management Authority | ndma.gov.in | Published guidance on crowd management at venues |
| MDN Web Docs | developer.mozilla.org | Geolocation behaviour and permission states used in alternate flow A1 |

*Table 8.1 Documentation and websites consulted*